



University
of Glasgow

**WORLD
CHANGING
GLASGOW**

Lecture 10

Embedded System Implementation: Serial Peripheral Interface (SPI)

UESTC 2004: Embedded Processors

Dr Mark D. Butala

Some slides were prepared by Dr Yao Sun,
Dr Lina Mohjazi, Dr Ahmad Taha, and
Dr Syed M. Raza





Aim

- Interface external components to the NUCLEO L432KC using SPI

Learning Objectives

By the end of this lesson you will be able to:

- Recall the characteristics of the SPI protocol
- Apply the SPI protocol on the ADXL345 accelerometer

Assumed Previous Knowledge

- Basic MCU interfacing techniques
- Basic understanding of serial communication techniques
- C/C++ programming



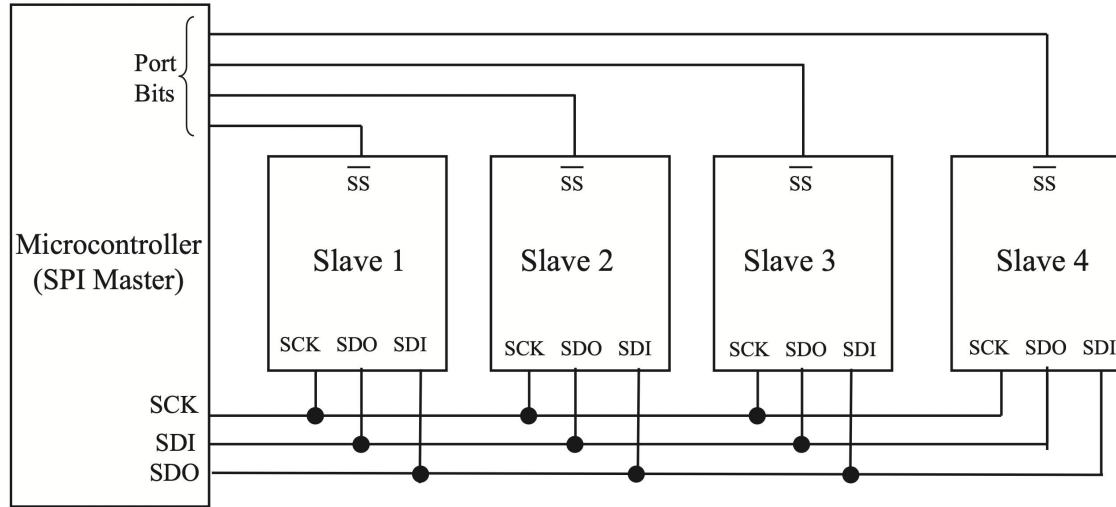
The Lab 3 manual is available on [Moodle](#).

Read it, **understand it**, and **prepare** yourself before you come to the lab.



- SPI is a **synchronous** communication protocol
- Invented in early **1980s** by Motorola
- Follows a **master/slave** hierarchy
- It uses four wires/lines between master and slave for **full duplex** communication:
 - ▶ Serial Data Out (SDO or MOSI)
 - ▶ Serial Data In (SDI or MISO)
 - ▶ Serial Clock (SCK)
 - ▶ Chip Select (CS) or Slave Select (SS)
- Master **establishes** and **control** all activity

Serial Peripheral Interface (SPI)



- **SDO line:** Transmits data from master to slave

- **SDI line:** Receives data from slave to master

- **SCK line:** Synchronizes data transmission between master and slave

- **CS or SS line:** Selects the slave device for communication

	System Firmware – Part 1
15	No Lecture
16	Lecture 12: Developing and Debugging Embedded System Firmware – Part 2

- SPI is suited to transferring data streams, e.g., data communication between microprocessors / ICs
- It can achieve higher data rates than I2C
- It doesn't send address or control information and doesn't include an acknowledge bit for every byte
- Data Transfer:
 - ▶ Master first configures the SPI clock (SCK) to be a frequency supported by the slave device
 - ▶ Master selects the slave device (Chip Select (CS) = LOW)
 - ▶ Master starts clock pulses indicating data is to be transferred
 - ▶ Master sends data as consecutive bits on MOSI line
- The number of bits in each data frame can be configured, usually between 4 and 16 bits

8	Lecture 7: Hardware/Software Interfacing Part 3: I2C, SPI, UART	Dr. Ahmad Taha
9	Lecture 8: Hardware/Software Interfacing Part 4: ADC	Dr. Ahmad Taha
10	Lecture 9: Hardware/Software Interfacing Part 5: DAC	Dr. Ahmad Taha
11	No Lecture	
12	No Lecture	
13	No Lecture	
14	Lecture 11: Developing and Debugging Embedded System Firmware – Part 1	Dr. Lina Mohjazi
15	No Lecture	
16	Lecture 12: Developing and Debugging Embedded System Firmware – Part 2	Dr. Lina Mohjazi

SPI Protocol Timing Diagram

MOSI: Master Out Slave In

MISO: Master In Slave Out

Start and End of Transmission

Sample

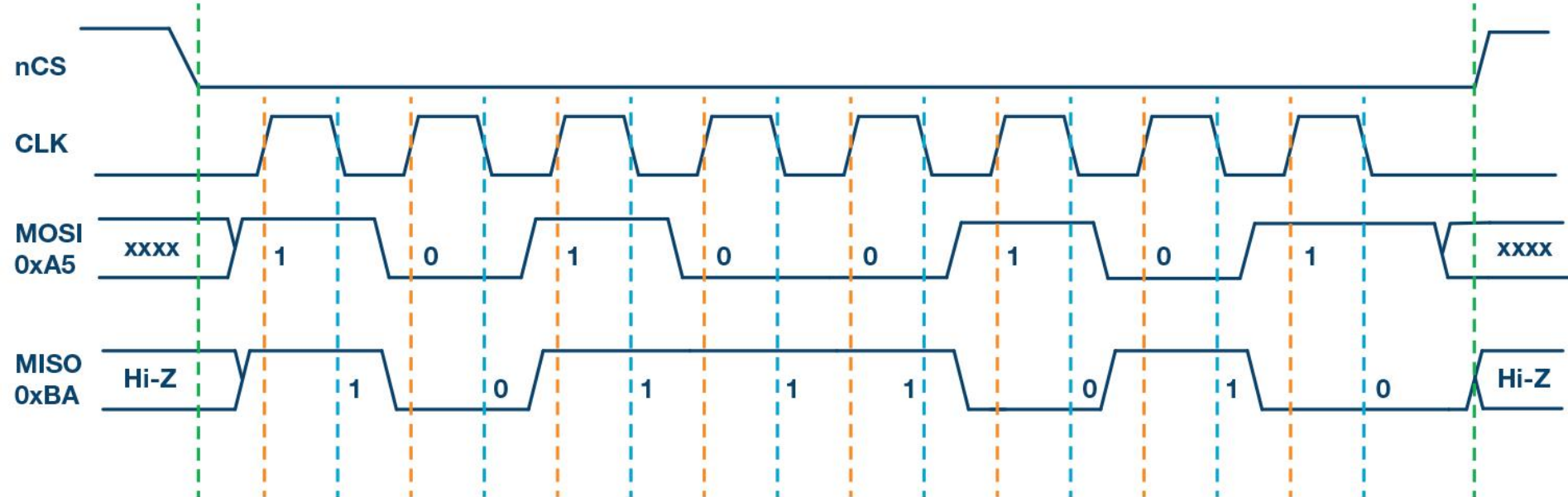
Shift

Lecture 7:

Hardware/Software Interfacing Part 3:

Dr. Ahmad Taha

I2C, SPI, USART



Clock Polarity and Phase

■ Clock Polarity (CPOL): CPOL bit sets polarity of the clock signal during the idle state

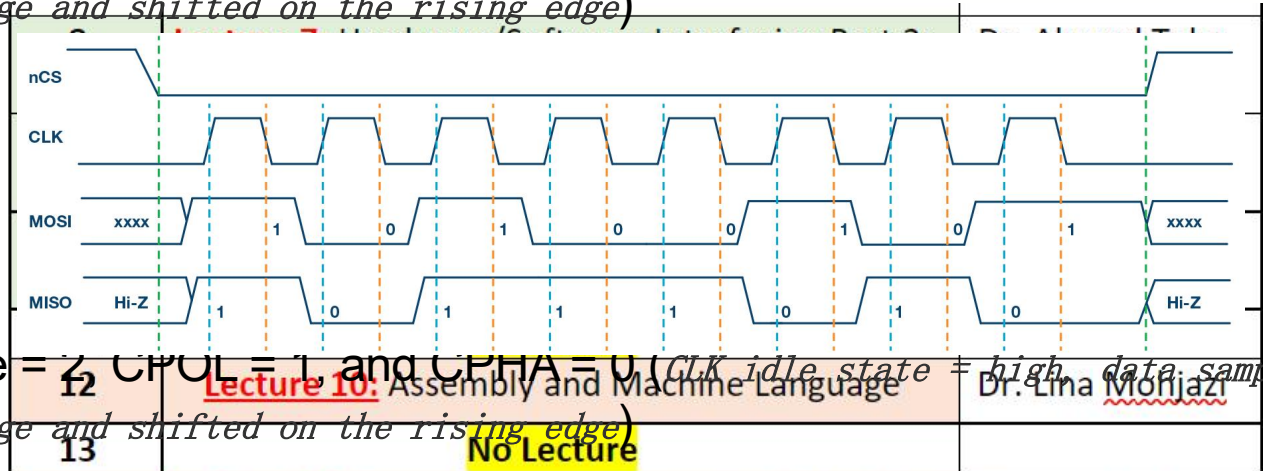
- ▶ The idle state: the period when CS is high and transition to low at the start of the transmission
- ▶ At the end of the transmission, CS transition from low to high

■ Clock Phase (CPHA): Depending on the CPHA bit, the **rising** or **falling** clock edge is used to sample and/or shift the data

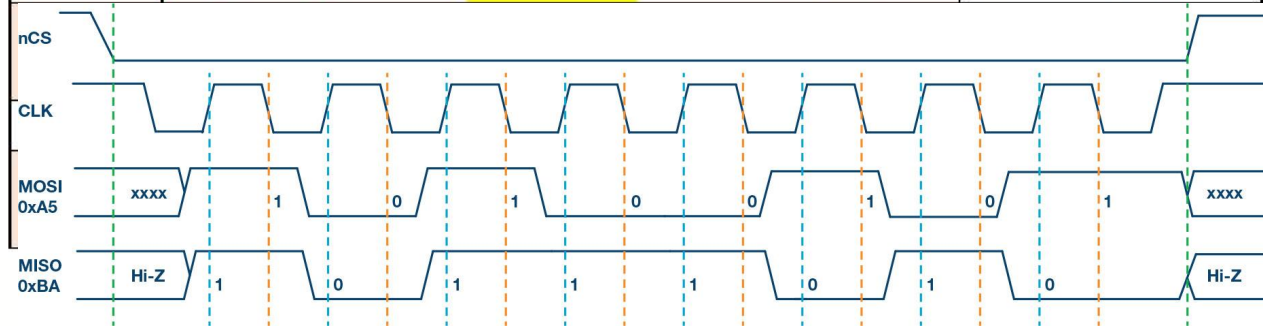
SPI Mode	CPOL	CPHA	Clock Polarity in Idle State	Clock Phase Used to Sample and/or Shift the Data
0	0	0	Logic low	Data sampled on rising edge and shifted out on the falling edge
1	0	1	Logic low	Data sampled on the falling edge and shifted out on the rising edge
2	1	0	Logic high	Data sampled on the falling edge and shifted out on the rising edge
3	1	1	Logic high	Data sampled on the rising edge and shifted out on the falling edge

SPI Characteristics – A Design Choice!

- SPI mode = 1, CPOL = 0, and CPHA = 1 (*CLK idle state = low, data sampled on the falling edge and shifted on the rising edge*)



- SPI mode = 2, CPOL = 1, and CPHA = 0 (*CLK idle state = high, data sampled on the falling edge and shifted on the rising edge*)



■ SPI advantages:

- ▶ Full-duplex communication
- ▶ Speed – I²C can go up to 5 Mbps while SPI can go with speed of 10 Mbps or more
- ▶ Not limited to 8-bit words
- ▶ Lower power requirements than I²C

■ I²C advantages:

- ▶ Simple – Implements a 2-wire interface unlike SPI
- ▶ Multiple slave devices without using extra pins
- ▶ Flexibility - Supports multiple Masters

8	<u>Lecture 7:</u> Hardware/Software Interfacing Part 3:	Dr. Ahmad Taha
9	<u>Lecture 8:</u> Hardware/Software Interfacing Part 4:	Dr. Ahmad Taha
10	<u>Lecture 9:</u> Hardware/Software Interfacing Part 5:	Dr. Ahmad Taha
11	No Lecture	
12	<u>Lecture 10:</u> Assembly and Machine Language	Dr. Lina <u>Mohjazi</u>
13	No Lecture	
14	<u>Lecture 11:</u> Developing and Debugging Embedded	Dr. Lina <u>Mohjazi</u>
15	No Lecture	
16	<u>Lecture 12:</u> Developing and Debugging Embedded	Dr. Lina <u>Mohjazi</u>
	System Firmware – Part 2	

Reminder: Why Learn SPI / I²C?

I²C Devices

SPI Devices

Acoustic sensor



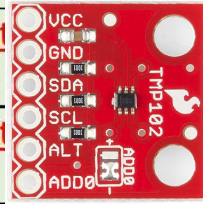
8

Temperature sensor

Lecture 7: Hardware/Software Interfacing
I²C, SPI, USART

9

Lecture 8:



Hardware/Software Interfacing
ADC

10

Lecture 9:

Hardware/Software Interfacing
DAC

Accelerometer

No Lecture

12

Lecture 10: Assembly and Mac

13

Thermal Imaging Sensor

No Lecture

14

Lecture 11: Developing and Debugging

System Firmware – Part 1

SD Card

15

No Lecture

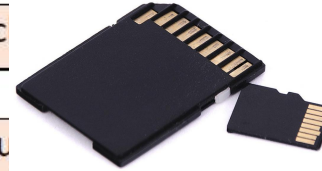
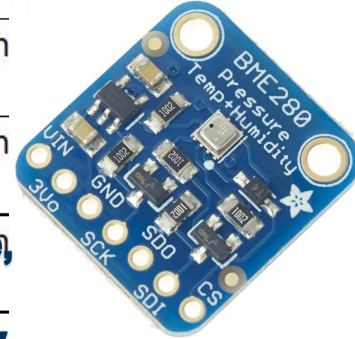
16

Developing and Debugging Embedded System Firmware – Part 2

Amad Tah

Amad Tah

Pressure, Temp, & Humidity Sensors



Lux sensor



-
- The diagram illustrates the pin connections for the NUCLEO-L432KC board. The connections are as follows:
- Pin 1 (PA_9):** D1, UART1_TX, PWM1/2, I2C1_SCL, VIN
 - Pin 2 (PA_10):** D0, UART1_RX, PWM1/3, I2C1_SDA, GND
 - Pin 3 (NRST):** NRST
 - Pin 4 (GND):** 5V
 - Pin 5 (PA_12):** D2, USB_DP, UART1_RTS, CAN1_TD, SPI1_MOSI, UART2_TX, A7, PA_2
 - Pin 6 (PB_0):** D3, ADC1/15, PWM1/2N, SPI1_SSEL, I2C3_SCL, PWM1/1N, SPI1_MOSI, ADC1/12, A6, PA_7
 - Pin 7 (PB_2):** D4, UART1_RX, I2C1_SDA, SPI1_MISO, PWM16/1, UART1_CTS, ADC1/11, A5, PA_6
 - Pin 8 (PB_6):** D5, UART1_TX, PWM16/1N, I2C1_SCL, SPI1_SCLK, DAC1/2, ADC1/10, A4, PA_5
 - Pin 9 (PB_1):** D6, UART1_RTS, PWM1/3N, SPI3_SSEL, DAC1/1, ADC1/9, A3, PA_4
 - Pin 10 (NC):** D7
 - Pin 11 (NC):** D8
 - Pin 12 (PA_8):** D9, PWM1/1, PWM1/2, UART2_RX, ADC1/8, A2, PA_3
 - Pin 13 (PA_11):** D10, CAN1_RD, USB_DM, UART1_CTS, PWM1/4, SPI1_MISO, SPI1_SCLK, PWM15/1N, UART2_RTS, ADC1/6, A1, PA_1
 - Pin 14 (PB_5):** D11, SPI3_MOSI, UART2_CTS, ADC1/5, A0, PA_0
 - Pin 15 (PB_4):** D12, UART1_CTS, I2C3_SDA, SPI3_MISO, SPI3_SCLK, UART1_RTS, LED1, D13, PB_3



Nucleo L432KC Board – SPI Functions (Master)



Function	Description
SPI Red_SPI (PinName mosi, PinName miso, PinName scl, PinName ssel, use_gpio_ssel_t)	Create an SPI Master interface, connected to the specified pins. This manipulates the SSEL pin using a DigitalOut object. This permits the use of <u><i>select()</i></u> and <u><i>deselect()</i></u> methods to keep the pin asserted between transfers.
void format (int bits, int mode=0)	Configure the data transmission <u>mode</u> and <u>data length</u>
void frequency (int hz=1000000)	Set the SPI bus clock frequency in <u>Hz</u>
int write (const char *tx_buffer, int tx_length, char *rx_buffer, int rx_length)	Write to an SPI Slave and return the response. The total number of bytes sent and received will be the maximum of <i>tx_length</i> and <i>rx_length</i> . The bytes written will be padded with the value <i>0xFF</i> .
void set_default_write_value (char data)	Set default write data. <u>SPI requires the master to send some data during a read operation.</u> Different devices may require different default byte values.

Function	Description
SPI_Slave SPI_Slave (PinName mosi, PinName miso, PinName sclk, PinName ssel)	Create an SPI Slave interface, connected to the specified pins
void format (int bits, int mode=0)	Configure the data transmission format
void frequency (int hz=1000000)	Set the SPI bus clock frequency
int receive (void)	Polls the SPI to see if data has been received
int read (void)	Retrieve data from receive buffer as Slave
void reply (int value)	Fill the transmission buffer with the value to be written out as Slave on the next received message from the Master.

SPI Example: Initialization

SPI Master

```
1  #include "mbed.h"
2
3  #define BLINK_DELAY 3000ms
4
5  const char TX_WORD          = 0xCA;
6  const char INCORRECT_TX_WORD = 0xA3;
7  const char EXPECTED_REPLY   = 0xFE;
8
9  int main() {
10     SPI SPI_master(A6, A5, A4); // MOSI, MISO, and SCLK
11     DigitalOut SPI_CS(D3);      // the "slave select" SS
12
13     SPI_master.format(8, 0);    // 8-bit data, Mode 0
14     SPI_master.frequency(1000000); // 1 MHz TX/RX rate
15
16     // Status LEDs
17     DigitalOut red_led(D12);
18     DigitalOut blink_led(LED1);
19
20     DigitalIn selector(D6);
21
22     int reply;
23
24     red_led = blink_led = 0;
```

SPI Slave

```
1  #include "mbed.h"
2
3  #define BLINK_DELAY 1000ms
4
5  const char TX_WORD          = 0xCA;
6  const char EXPECTED_REPLY   = 0xFE;
7  const char INCORRECT_REPLY = 0x4B;
8
9  int main() {
10     SPISlave SPI_slave(A6, A5, A4, D3); // MOSI, MISO, SCLK, and SS
11
12     SPI_slave.format(8, 0);             // 8-bit data, Mode 0
13     SPI_slave.frequency(1000000);      // 1 MHz TX/RX rate
14
15     // Status LEDs
16     DigitalOut green_led(D11);
17     DigitalOut red_led(D12);
18     DigitalOut blink_led(LED1);
19
20     DigitalIn selector(D6);
21
22     char rx_value;
23
24     green_led = red_led = blink_led = 0;
```

SPI Example: *while(1)* Loop

SPI Master

```

26 while (1) {
27     ThisThread::sleep_for(BLINK_DELAY);
28
29     SPI_CS = 0;
30     reply = SPI_master.write(selector ?
31                               TX_WORD :
32                               INCORRECT_TX_WORD);
33     SPI_CS = 1;
34
35     printf("reply = 0x%X\n", reply);
36     if (reply == EXPECTED_REPLY) {
37         red_led = 1;
38     }
39     else {
40         red_led = 0;
41     }
42     blink_led = !blink_led;
43 }
44

```

```

221 ✓ int SPI::write(int value)
222 {
223     select();
224     int ret = spi_master_write(&_peripheral->spi, value);
225     deselect();
226     return ret;
227 }

```

SPI Slave

```

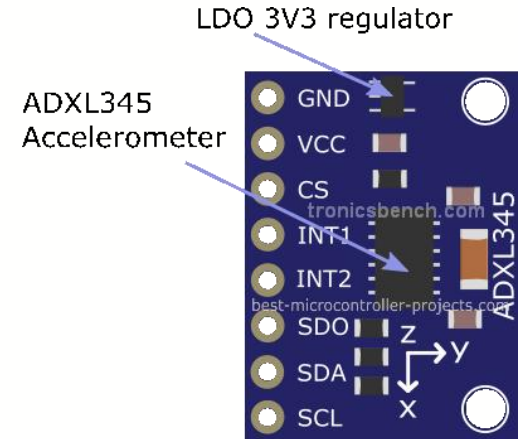
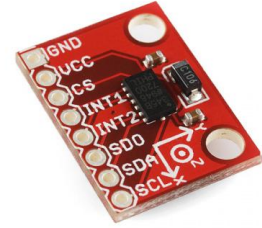
26 while (1) {
27     ThisThread::sleep_for(BLINK_DELAY);
28
29     if (SPI_slave.receive()) {
30         green_led = 1;
31         rx_value = SPI_slave.read();
32         SPI_slave.reply(selector ?
33                           EXPECTED_REPLY :
34                           INCORRECT_REPLY);
35         printf("rx_value = 0x%X\n", rx_value);
36         if (rx_value == TX_WORD) {
37             red_led = 1;
38         }
39         else {
40             red_led = 0;
41         }
42     }
43     else {
44         green_led = red_led = 0;
45     }
46     blink_led = !blink_led;
47 }
48
49

```

The *write()* function in the mbed library asserts and de-asserts the CS. In the exam you still **MUST** include it in your code.

ADXL345 Accelerometer

- Interface the ADXL345 accelerometer to the NUCLEO L432KC mbed board using SPI
- We will need to configure the SPI on our mbed board in accordance with the accelerometer requirements
- We will also configure the accelerometer at the beginning of our C code



ADXL345 Accelerometer

- Things to consider prior to implementation:
 - ▶ Interface type – in this case SPI and I²C (Design Choice!!!)
- Voltage levels
- Software configuration
- All those details can be found in the **module's datasheet**

Address	Name	Description
0x00	DEVID	Device ID
0x1D	THRESH_TAP	Tap threshold
0x1E/1F/20	OFSX, OFSY, OFSZ	X, Y, Z axis offsets
0x21	DUR	Tap duration
0x2D	POWER_CTL	Power-saving features control. Device powers up in standby mode; setting bit 3 causes it to enter Measure mode.
0x31	DATA_FORMAT	Data format control Bits 7: force a self-test by setting to 1 6: 1 = three-wire SPI mode; 0 = four-wire SPI mode 5: 0 sets interrupts active high, 1 sets them active low 4: always 0 3: 0 = output is 10-bit always; 1 = output depends on range setting 2: 1 = left justify result; 0 = right justify result 1-0: 00 = $\pm 2g$; 01 = $\pm 4g$; 10 = $\pm 8g$; 11 = $\pm 2g$
0x33:0x32	DATA1:DATA0	X Axis Data, formatted according to DATA_FORMAT, in 2's complement.
0x35:0x34	DATAY1:DATAY0	Y Axis Data, as above
0x37:0x36	DATAZ1:DATAZ0	Z Axis Data, as above

^aIn any data transfer the register address is sent first, and formed:

bit 7 = R/W (1 for read, 0 for write); bit 6: 1 for multiple byte, 0 for single;
bits 5-0: the lower 5 bits found in the Address column.



ADXL345 Accelerometer

- Max clock speed 5MHz
- SPI bit 1 for 3 wire setup
- CPOL =1 and CPHA = 1
- MB bit = 1 for multi-bytes in one transmission

Register 0x31—DATA_FORMAT (Read/Write)

D7	D6	D5	D4	D3	D2	D1	D0
SELF_TEST	SPI	INT_INVERT	0	FULL_RES	Justify	Range	

REGISTER MAP

Table 19.

Address		Name	Type	Reset Value	Description
Hex	Dec				
0x00	0	DEVID	R	11100101	Device ID
0x01 to 0x1C	1 to 28	Reserved			Reserved; do not access
0x1D	29	THRESH_TAP	R/W	00000000	Tap threshold
0x1E	30	OFSX	R/W	00000000	X-axis offset
0x1F	31	OFSY	R/W	00000000	Y-axis offset
0x20	32	OFSZ	R/W	00000000	Z-axis offset
0x21	33	DUR	R/W	00000000	Tap duration
0x22	34	Latent	R/W	00000000	Tap latency
0x23	35	Window	R/W	00000000	Tap window
0x24	36	THRESH_ACT	R/W	00000000	Activity threshold
0x25	37	THRESH_INACT	R/W	00000000	Inactivity threshold
0x26	38	TIME_INACT	R/W	00000000	Inactivity time
0x27	39	ACT_INACT_CTL	R/W	00000000	Axis enable control for activity and inactivity detection
0x28	40	THRESH_FF	R/W	00000000	Free-fall threshold
0x29	41	TIME_FF	R/W	00000000	Free-fall time
0x2A	42	TAP_AXES	R/W	00000000	Axis control for single tap/double tap
0x2B	43	ACT_TAP_STATUS	R	00000000	Source of single tap/double tap
0x2C	44	BW_RATE	R/W	00001010	Data rate and power mode control
0x2D	45	POWER_CTL	R/W	00000000	Power-saving features control
0x2E	46	INT_ENABLE	R/W	00000000	Interrupt enable control
0x2F	47	INT_MAP	R/W	00000000	Interrupt mapping control
0x30	48	INT_SOURCE	R	00000010	Source of interrupts
0x31	49	DATA_FORMAT	R/W	00000000	Data format control
0x32	50	DATA0	R	00000000	X-Axis Data 0
0x33	51	DATA1	R	00000000	X-Axis Data 1
0x34	52	DATA0	R	00000000	Y-Axis Data 0
0x35	53	DATA1	R	00000000	Y-Axis Data 1
0x36	54	DATA0	R	00000000	Z-Axis Data 0
0x37	55	DATA1	R	00000000	Z-Axis Data 1
0x38	56	FIFO_CTL	R/W	00000000	FIFO control
0x39	57	FIFO_STATUS	R	00000000	FIFO status



ADXL345: SPI Read/Write Timing Diagram

4-Wire Timing Diagram

Writing

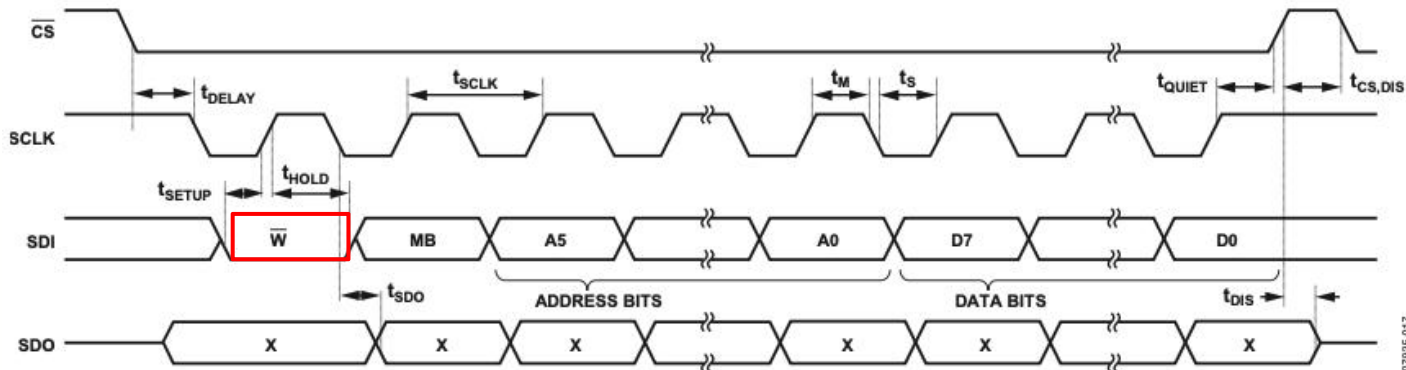


Figure 37. SPI 4-Wire Write

Reading

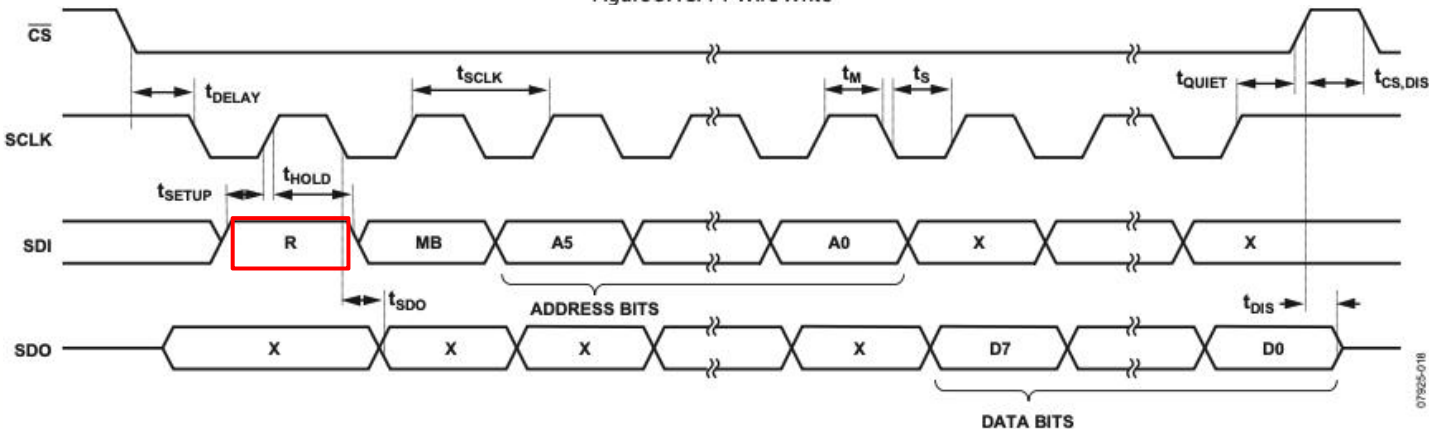


Figure 38. SPI 4-Wire Read

```

1  #include "mbed.h"
2
3  #define FULL_RES_SCALE_FACTOR 0.004 // g / LSB
4
5  #define READ 0x80
6  #define MULTIPLE_BYTE_TX 0x40
7
8  // MOSI (SDA), MISO (SD0), and SCLK (SCL)
9  SPI axl345(D2, D10, A4);
10 DigitalOut axl345_CS(D3); // SPI chip select
11
12 void initialize() {
13     axl345.format(8, 3);
14     axl345.frequency(2000000);
15
16     // Set data format register
17     axl345_CS = 0;
18     axl345.write(0x31);
19     axl345.write(0x0B);
20     axl345_CS = 1;
21
22     // Set power control register
23     axl345_CS = 0;
24     axl345.write(0x2D);
25     axl345.write(0x08);
26     axl345_CS = 1;
27 }

```

0x0B = 00001011

Register 0x31—DATA_FORMAT (Read/Write)

D7	D6	D5	D4	D3	D2	D1	D0
SELF_TEST	SPI	INT_INVERT	0	FULL_RES	Justify	Range	

FULL_RES Bit

When this bit is set to a value of 1, the device is in full resolution mode, where the output resolution increases with the *g* range set by the range bits to maintain a 4 mg/LSB scale factor. When the FULL_RES bit is set to 0, the device is in 10-bit mode, and the range bits determine the maximum *g* range and scale factor.

DATAx1 REGISTER

D7	D6	D5	D4	D3	D2	D1	D0
D7	D6	D5	D4	D3	D2	D1	D0

DATAx0 REGISTER

D7	D6	D5	D4	D3	D2	D1	D0
D7	D6	D5	D4	D3	D2	D1	0

Read
datasheet for
more details

Right-justified

```

1  #include "mbed.h"
2
3  #define FULL_RES_SCALE_FACTOR 0.004 // g / MSB
4
5  #define READ 0x80
6  #define MULTIPLE_BYTE_TX 0x40
7
8  // MOSI (SDA), MISO (SD0), and SCLK (SCL)
9  SPI axl345(D2, D10, A4);
10 DigitalOut axl345_CS(D3); // SPI chip select
11
12 void initialize() {
13     axl345.format(8, 3);
14     axl345.frequency(2000000);
15
16     // Set data format register
17     axl345_CS = 0;
18     axl345.write(0x31);
19     axl345.write(0x0B);
20     axl345_CS = 1;
21
22     // Set power control register
23     axl345_CS = 0;
24     axl345.write(0x2D);
25     axl345.write(0x08);
26     axl345_CS = 1;
27 }

```

0x08 = 00001000

Register 0x2D—POWER_CTL (Read/Write)

D7	D6	D5	D4	D3	D2	D1	D0
0	0	Link	AUTO_SLEEP	Measure	Sleep	Wakeup	

Measure Bit

A setting of 0 in the measure bit places the part into standby mode, and a setting of 1 places the part into measurement mode. The [ADXL345](#) powers up in standby mode with minimum power consumption.

**Read
datasheet for
more details**

```

29 int main() {
30     DigitalOut led(LED1);
31
32     initialize();
33
34     char buffer[6];
35     int16_t data[3];
36     float x, y, z;
37
38     while (1) {
39         ThisThread::sleep_for(200ms);
40
41         axl345_CS = 0;
42         axl345.write(READ | MULTIPLE_BYTE_TX | 0x32);
43
44         for (unsigned i = 0; i < 6; i++) {
45             buffer[i] = axl345.write(0x00);
46         }
47         axl345_CS = 1;
48
49         // combine MSB and LSB
50         data[0] = buffer[1]<<8 | buffer[0];
51         data[1] = buffer[3]<<8 | buffer[2];
52         data[2] = buffer[5]<<8 | buffer[4];
53
54         x = FULL_RES_SCALE_FACTOR*data[0];
55         y = FULL_RES_SCALE_FACTOR*data[1];
56         z = FULL_RES_SCALE_FACTOR*data[2];
57
58         // Output x,y,z accelerations in units of g
59         printf("x = %+1.2fg\t y = %+1.2fg\t z = %+1.2fg\n\r", x, y, z);
60
61         led = !led;
62     }
63 }

```

0x32	50	DATA0	R	00000000	X-Axis Data 0
0x33	51	DATA1	R	00000000	X-Axis Data 1
0x34	52	DATA0	R	00000000	Y-Axis Data 0
0x35	53	DATA1	R	00000000	Y-Axis Data 1
0x36	54	DATA0	R	00000000	Z-Axis Data 0
0x37	55	DATA1	R	00000000	Z-Axis Data 1

$$0x80 | 0x40 | 0x32 = 0xF2 \\ = 11110010$$

To read or write multiple bytes in a single transmission, the multiple-byte bit, located after the R/W bit in the first byte transfer (MB in Figure 37 to Figure 39), must be set. After the register addressing and the first byte of data, each subsequent set of clock pulses (eight clock pulses) causes the ADXL345 to point to the next register for a read or write. This shifting continues until the clock pulses cease and CS is deasserted. To perform reads or writes on different, nonsequential registers, CS must be deasserted between transmissions and the new register must be addressed separately.

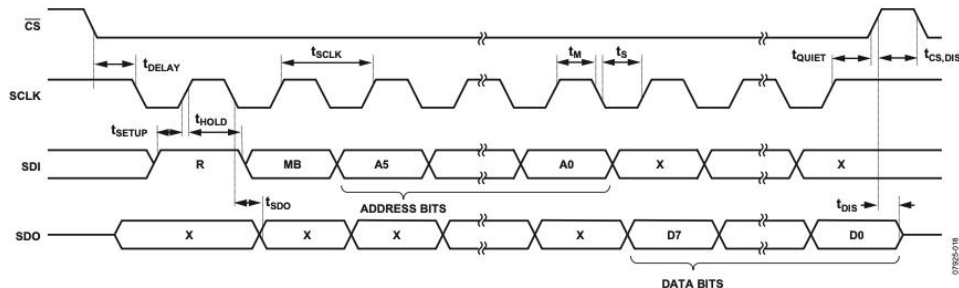


Figure 38. SPI 4-Wire Read

```

29  int main() {
30      DigitalOut led(LED1);
31
32      initialize();
33
34      char buffer[6];
35      int16_t data[3];
36      float x, y, z;
37
38      while (1) {
39          ThisThread::sleep_for(200ms);
40
41          axl345_CS = 0;
42          axl345.write(READ | MULTIPLE_BYTE_TX | 0x32);
43
44          for (unsigned i = 0; i < 6; i++) {
45              buffer[i] = axl345.write(0x00);
46          }
47          axl345_CS = 1;
48
49          // combine MSB and LSB
50          data[0] = buffer[1]<<8 | buffer[0];
51          data[1] = buffer[3]<<8 | buffer[2];
52          data[2] = buffer[5]<<8 | buffer[4];
53
54          x = FULL_RES_SCALE_FACTOR*data[0];
55          y = FULL_RES_SCALE_FACTOR*data[1];
56          z = FULL_RES_SCALE_FACTOR*data[2];
57
58          // Output x,y,z accelerations in units of g
59          printf("x = %+1.2fg\t y = %+1.2fg\t z = %+1.2fg\n\r", x, y, z);
60
61          led = !led;
62      }
63  }

```

DATAx1 REGISTER

D7	D6	D5	D4	D3	D2	D1	D0
D7	D6	D5	D4	D3	D2	D1	D0

DATAx0 REGISTER

D7	D6	D5	D4	D3	D2	D1	D0
D7	D6	D5	D4	D3	D2	D1	0

Right-justified



Summary

- Characteristics and details of the SPI protocols
- Extracting information from the slave devices' datasheets (ADXL345)
- Implementing the SPI protocols to interface the ADXL345

Review Exercise

- You can try different configurations of the sensor